Project- High Level Design

On

Finance Support Triage Agent

Course Name: Agentic AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	Tarang Purohit	
2	Suhani Maheshwari	EN22CS301988
3	Smita Kumawat	EN22CS301961
4	Somya Neema	EN22CS3019
5	Sneha Agrawal	EN22CS3019

Group Name: 03D9

Project Number: AAI-03

Industry Mentor Name: Mr. Ashwin

University Mentor Name :Prof. Pradeep Baniya

Academic Year:2025-26

Table of Contents

1. Introduction.
   1.1. Scope of the document.
   1.2. Intended Audience
   1.3. System overview.

2. System Design.
   2.1. Application Design
   2.2. Process Flow.
   2.3. Information Flow.
   2.4. Components Design
   2.5. Key Design Considerations
   2.6. API Catalogue.

3. Data Design.
   3.1. Data Model
   3.2. Data Access Mechanism
   3.3. Data Retention Policies
   3.4. Data Migration

4. Interfaces

5. State and Session Management

6. Caching

7. Non-Functional Requirements
   7.1. Security Aspects
   7.2. Performance Aspects

8. References

1. Introduction

1.1 Scope of the Document

This High Level Design (HLD) document describes the complete architectural and technical design of the Finance Support Triage AI System (FSTAS). The system is a backend-first, AI-powered customer support platform designed to automatically classify, analyse, and respond to finance-related user queries across three primary domains: fraud detection and reporting, refund and reversal processing, and payment failure diagnostics.

This document defines:

- The overall system architecture spanning the presentation, API, orchestration, inference, and data layers.
- The multi-agent CrewAI pipeline design, including each agent's role, input, output, and interaction protocol.
- The FastAPI REST interface specifications, including all endpoints, request/response schemas, and error contracts.
- The MongoDB data model, including collection schemas, indexing strategy, and data retention policies.
- Non-functional requirements including security, performance, scalability, and fault tolerance.
- Caching and session management strategies.

Out of scope: Low-level code implementation, DevOps provisioning templates, regulatory compliance auditing procedures, and third-party vendor SLA management

1.2 Intended Audience

This document is targeted at the following roles:

Role	Relevance
Solution / System Architects	Validate architecture patterns, scalability choices, and integration design.
Backend Engineers (Python)	Implement FastAPI routes, CrewAI agents, MongoDB repositories.
Frontend Engineers (React)	Understand API contracts, response structure, and session behaviour.
QA / Test Engineers	Design integration, regression, and load test suites.
Security Engineers	Review authentication flows, data protection, and threat surface.
Product Managers	Understand system capabilities, constraints, and user-facing behaviour.
DevOps / SRE Teams	Plan deployment topology, monitoring, and incident response.

1.3 System Overview

The Finance Support Triage AI System (FSTAS) automates the first-line analysis of finance-related customer support queries. Instead of routing every query to a human agent for initial triage, FSTAS employs a coordinated pipeline of five AI agents — each specialised in a specific reasoning task — to produce a structured, decision-ready output within seconds.

Key operational characteristics:

- Query domains supported: fraud alerts, refund requests, payment failures, transaction disputes, account holds.
- Multi-agent pipeline: each query passes through a Classifier, Extractor, Decision, Validator, and Responder agent sequentially.
- LLM backbone: Google Gemini APIpowers each agent's natural language reasoning.

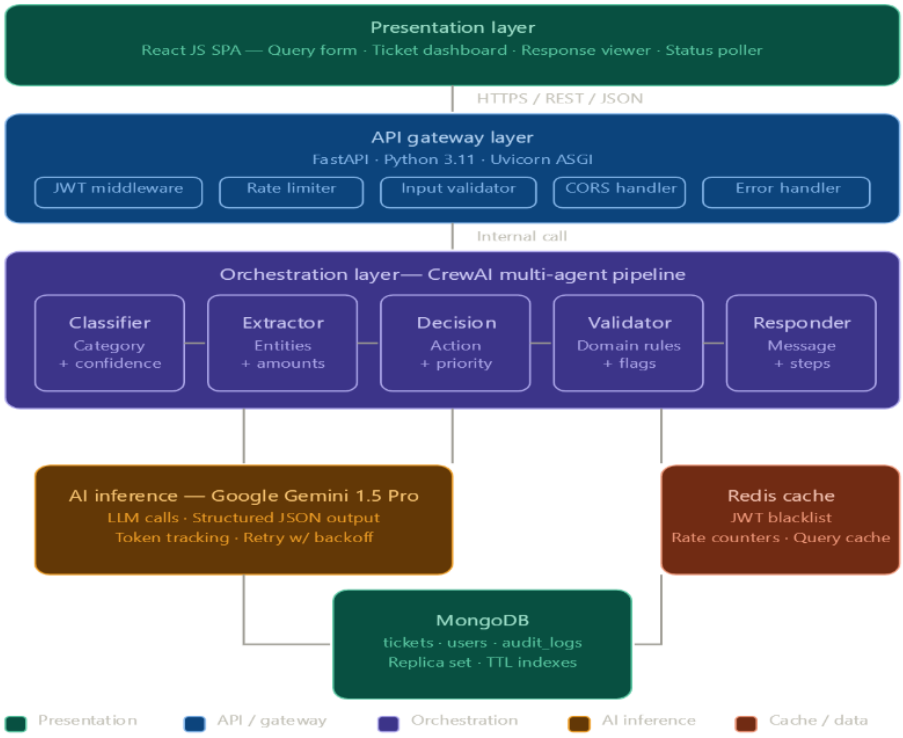
- Data persistence: all inputs, intermediate agent outputs, decisions, and responses are persisted to MongoDB for auditability and analytics.
- REST-first API: all functionality is exposed via versioned FastAPI endpoints consumable by the React frontend or any third-party integration.
- Async-ready: long-running queries are processed asynchronously using background tasks with status polling.

The system is purpose-built to reduce average first-response time from hours (human triage) to under 10 seconds (AI triage), while maintaining a complete audit trail for regulatory compliance.

2. System Design

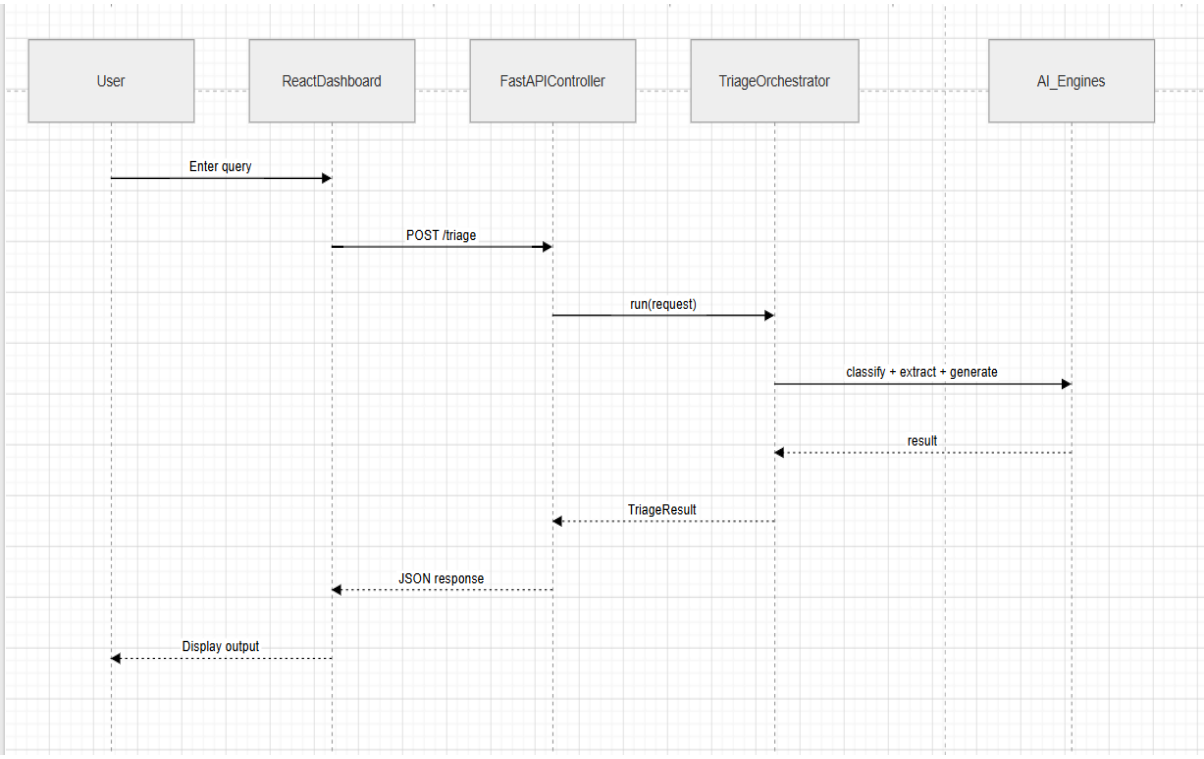
2.1 Application Design

FSTAS is structured as a five-layer application following a clean separation of concerns. Each layer has a well-defined responsibility boundary and communicates with adjacent layers through formal interfaces.



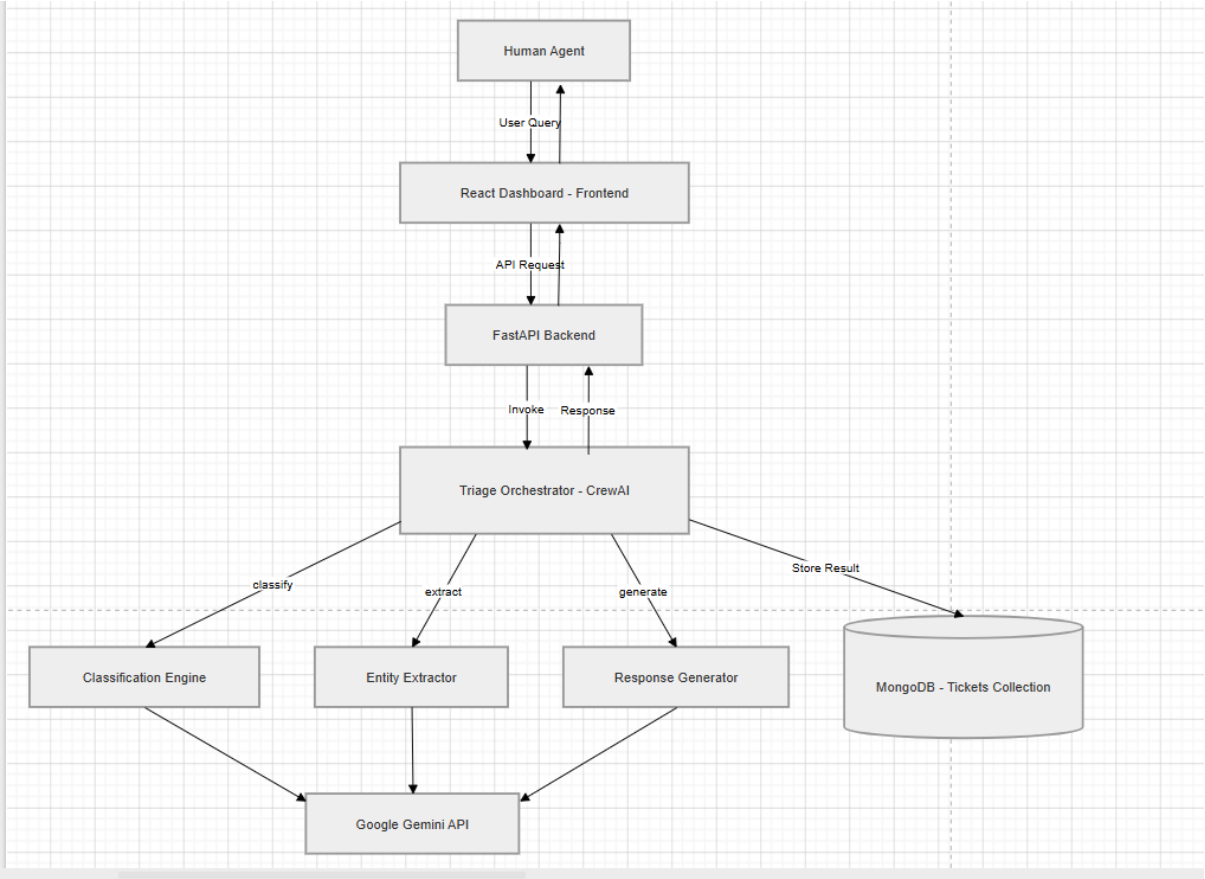
2.2 Process Flow

The end-to-end process flow for a user-submitted finance query is described below, covering both the synchronous happy path and the asynchronous fallback path.



2.3 Information Flow

The information flow diagram illustrates how data is transformed as it traverses each layer and each agent stage. Each transformation step enriches the context object, which is passed cumulatively through the pipeline.



2.4 Components Design

This section describes the major components, their responsibilities, and interactions.

2.4.1 React Frontend

- Built using React (SPA architecture)
- Communicates with backend via REST APIs
- Key modules:
  - **QueryForm** → submits user queries
  - **TicketDashboard** → displays tickets with filters
  - **ResponseViewer** → shows AI-generated structured response
  - **StatusPoller** → polls ticket status for async processing

- **AuthContext** → manages JWT authentication

2.4.2 FastAPI Backend

- Central API layer handling all requests
- Responsibilities:
  - Request validation (Pydantic)
  - Authentication (JWT)
  - Routing (/api/v1/*)
  - Background task execution (CrewAI pipeline)
- Includes middleware for:
  - CORS, logging, rate limiting, error handling
- Uses dependency injection for DB, config, and services

2.4.3 CrewAI Orchestration Layer

- Manages multi-agent pipeline (sequential execution)

Agents & Roles:

Agent	Responsibility	Output
Classifier	Categorize query	category
Extractor	Extract financial data	amount, date, txn_id
Decision	Determine action & risk	action, risk_level
Validator	Validate & fix outputs	validated JSON
Responder	Generate user response	message

2.4.4 Gemini LLM Integration

- Uses Google Gemini API
- Each agent sends structured prompts with JSON output format
- Configuration:

  - Low temperature for accuracy (Classifier, Extractor, Validator)
  - Moderate for response generation
- Features:
  - JSON output enforcement
  - Retry mechanism for failures
  - Token usage tracking

2.5 Key Design Considerations

Scalability

- Backend is **stateless**, enabling horizontal scaling behind load balancers
- Shared infrastructure:
  - MongoDB (replica set)
  - Redis (for caching, future use)
- CrewAI pipeline runs as background tasks
- For high load:
  - Can offload processing to **worker queue (Celery + RabbitMQ)**
- MongoDB supports read scaling via secondary nodes

Fault Tolerance

- Each agent has **retry mechanism (max 3 retries)**
- On failure:
  - Ticket marked as **PARTIAL_FAILURE**
  - Error logged with failed agent info

- Fallback response ensures **graceful degradation**
- No raw exceptions exposed to client → always structured response

Modularity

- Agents are **independent and loosely coupled**
- New agents can be added without changing:
  - API layer
  - Database layer
- CrewAI pipeline is easily extendable (add new task + agent)
- MongoDB schema is flexible → no rigid schema migrations required

2.6 API Catalogue

All APIs are versioned under:

```
/api/v1/
```

1. Create Triage Request

POST /api/v1/triage

Description:
Submit a finance-related query for AI processing.

Request

```
{
  "query": "I was charged 3000 INR twice yesterday"
}
```

Response (202 Accepted - async)

```
{
  "ticket_id": "abc123",
  "status": "PROCESSING",
  "message": "Your request is being processed"
}
```

---

2. Get Triage Result

GET /api/v1/tickets/{ticket_id}

Description:
Fetch processed result for a specific ticket.

Response

```
{
  "ticket_id": "abc123",
  "status": "COMPLETED",
  "data": {
  "category": "payment_issue",
  "entities": {
  "amount": "3000 INR",
  "date": "yesterday",
  "transaction_id": ""
 },
  "decision": {
  "risk_level": "high",
  "action": "investigate"
 }
 },
  "response": "We understand you were charged twice and are currently
reviewing your transaction. Our team will resolve this issue shortly."
}
```

---

3. Get All Tickets

GET /api/v1/tickets

Description:
Fetch all user tickets (with optional filters).

Query Params (optional)

```
?status=COMPLETED
?category=fraud
?page=1&limit=10
```

Response

```
[
  {

"ticket_id": "abc123",
    "query": "...",
    "status": "COMPLETED",
    "created_at": "2026-03-22T10:00:00Z"
  }
]
```

3. Data Design

3.1 Data Model

The system uses **MongoDB** with a document-based schema optimized for flexible AI outputs.

Collections:

- **tickets** → stores full query lifecycle
- **users** → authentication & roles
- **audit_logs** → system events tracking

---

tickets (Main Collection)

Stores query, AI outputs, and final response.

```
{
 "ticket_id": "string",
 "user_id": "string",
 "query_text": "string",
 "status": "PROCESSING | COMPLETED | FAILED",

 "category": "string",
 "priority": "HIGH",
 "risk_level": "LOW",

 "extracted_entities": {
  "amount": 249.99,
```

```
  "date": "2025-05-15"
 },

 "decision": {
  "recommended_action": "INITIATE_REFUND"
 },

 "response": {
  "message": "string"
 },

 "agent_outputs": [],
 "created_at": "datetime"
}
```


users


```
{
 "user_id": "string",
 "email": "string",
 "password_hash": "string",
 "role": "USER | ADMIN"
}
```


3.2 Data Access Mechanism

- Uses **Repository Pattern**
  → separates database logic from business logic
- Uses **async MongoDB driver (Motor)**
  → non-blocking operations
- Uses connection pooling
  → improves performance under load
- Transactions used for:
  - updating ticket
  - writing logs

4. Interfaces

4.1 External Interfaces

4.1.1 Google Gemini API

- **Type:** REST API (HTTPS)
- **SDK:** google-generativeai (Python)
- **Endpoint:**

https://generativelanguage.googleapis.com/

- **Authentication:** API Key (GOOGLE_API_KEY)
- **Model Used:** gemini-1.5-pro
- **Timeout:** 30 seconds per request

Behavior

- Each AI agent sends structured prompts to Gemini
- Expects **strict JSON output**
- Retry mechanism:
  - Handles **429 (rate limit)**
  - Handles **5xx errors**
  - Uses exponential backoff (1s → 2s → 4s)

4.1.2 Frontend ↔ Backend (React → FastAPI)

- **Type:** REST API over HTTPS
- **Data Format:** JSON
- **Authentication:** Bearer Token (JWT)

Key Features

- CORS restricted to frontend domain
- All responses include:
  - request_id → for tracing/debugging
- Error handling follows **standard format**:

4.2 Internal Interfaces

4.2.1 FastAPI to CrewAI Pipeline

Interface type: In-process Python function call. The TriageCrew class exposes a single kickoff(query_input: QueryInput) -> TriageResult method. This is called from a FastAPI BackgroundTask. Results are written directly to MongoDB by the pipeline, and the API layer polls MongoDB for status updates on client request.

4.2.2 CrewAI Agents to MongoDB

Agents do not interact with MongoDB directly. All persistence is handled by the TriageCrew orchestrator after each agent task completes. This keeps agents stateless and focused on their reasoning task.

5. State and Session Management

5.1 User Session Management

The system follows a **stateless architecture**, meaning no session data is stored on the server.
All user authentication and session handling is managed using **JWT (JSON Web Tokens)**

Token Types

- **Access Token**
  - Short-lived (1 hour)
  - Contains: user_id, role,
  - Used for all authenticated API requests

- **Refresh Token**
  - Long-lived (7 days)
  - Used to generate new access tokens without login

Token Storage Strategy

- Access Token → stored in **React memory (not localStorage)**
- Refresh Token → stored in **HttpOnly, Secure cookie**

5.2 Pipeline State Management

The AI pipeline uses a **shared context object (TriageContext)** that flows through all agents.

How It Works

Each agent:

- Reads previous outputs
-
- Adds its own result to context

Failure Handling

- If any agent fails:
  - Pipeline stops or continues with fallback
  - Partial results are saved in MongoDB
  - Failed agent name + error stored

6. Caching Strategy

The system uses caching to improve performance and reduce repeated LLM calls.

• Frequently accessed tickets cached using Redis
• API responses cached for short duration
• Future scope: semantic caching for repeated queries

7. Non-Functional Requirements

- **7.1 Security Aspects:**
  - **Authentication/Authorization:** OAuth2 with JWT access tokens and role-based access control (RBAC) for admin endpoints.
  - **Encryption:** TLS for all in-transit; MongoDB at-rest encryption; field-level encryption for PII; encryption keys stored in KMS (AWS KMS / GCP KMS).
  - **PII Handling:** Redaction and tokenization before sending to LLMs. Option to store raw PII only when explicitly required and encrypted.
  - **Least Privilege:** Service accounts for external integrations, scoped secrets. Use Vault for secret management.
  - **Audit Logging:** Immutable audit trails for policy decisions and admin actions. Retain logs per retention policy.
  - **Securing LLM calls:** Sanitize prompts; do not send more data than necessary to Google Gemini; use content filters and an approval flow for prompt/template changes

7.2 Performance Aspects:

- **Latency Targets:**
  - API 95th percentile response time for triage submission: ≤ 250 ms for acceptance ack; final triage result: < 3s for typical cases (subject to LLM latency). For complex cases with external validation, accept asynchronous processing and return 202 with webhook/callback.

- **Throughput:**
  - Design target: 200 requests/sec initially, scaleable to 2000 RPS. Plan LLM call concurrency limits to avoid rate limits.
- **Scaling Strategy:**
  - Stateles FastAPI instances behind load balancer; autoscale based on CPU and request latency.
  - CrewAI and agents horizontally scaled; use Kubernetes HPA or serverless autoscaling.

8. References

- FastAPI — https://fastapi.tiangolo.com
- CrewAI (or orchestration docs) — vendor docs for CrewAI orchestration patterns.
- Google Gemini API docs — (internal/public docs for LLM usage and quota patterns).
- MongoDB schema design & sharding — https://www.mongodb.com/docs/manual/